

# Pipelines: A Complete Guide

## What Is a Pipeline?

A **Pipeline** in scikit-learn is a way to chain multiple steps—such as **preprocessing, feature selection, and modeling**—into a **single, unified object**.

## Why use a pipeline?

- Ensures **reproducibility** and **clean code**
- Prevents **data leakage** (e.g., fitting scalers only on training data)
- Works seamlessly with `GridSearchCV`, **cross-validation**, and `fit / predict` methods

## 1. Basic Structure of a Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),      # Step 1: Scaling
    ('clf', LogisticRegression())     # Step 2: Classifier
])
```

- Each step is a **name-object pair**.
- All steps except the last must be **transformers** (i.e., implement `fit` and `transform`).
- The last step can be **estimator or transformer**.

## 2. Using Pipeline

```
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
```

That's it! The pipeline handles:

- Fitting the scaler on `X_train`
- Transforming `X_train` with it
- Training the classifier
- Automatically applying the same scaling to `X_test`

### 3. Accessing Steps in Pipeline

```
pipe.named_steps['scaler'] # Get the scaler  
pipe.named_steps['clf']   # Get the model
```

You can also slice parts of a pipeline:

```
pipe[:-1] # All but last step  
pipe[-1]  # Just the final estimator
```

### 4. Pipelines with Column-Specific Transformers

Use `ColumnTransformer` to apply different preprocessing to different columns.

```
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import OneHotEncoder  
from sklearn.impute import SimpleImputer  
  
preprocessor = ColumnTransformer([  
    ('num', StandardScaler(), ['age', 'income']),  
    ('cat', OneHotEncoder(), ['gender', 'city'])  
)  
  
pipe = Pipeline([  
    ('prep', preprocessor),  
    ('clf', LogisticRegression())  
)
```

## 5. Custom Steps in a Pipeline

You can write your own transformer by extending `BaseEstimator` and `TransformerMixin`.

```
from sklearn.base import BaseEstimator, TransformerMixin

class AddFeature(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        X = X.copy()
        X['new_feature'] = X['feature1'] * X['feature2']
        return X
```

Then plug it into your pipeline.

### Challenge

Try building a pipeline that:

- Imputes missing values
- Scales numerical features
- Encodes categorical features
- Uses `RandomForestClassifier`
- Uses `GridSearchCV` for `max_depth` and `n_estimators`